PROGRAMACIÓN ORIENTADA A OBJETOS

1. ¿QUÉ ES UN PARADIGMA DE PROGRAMACIÓN?

Es la forma en que los programadores decidimos como irá estructurado nuestro código. Un paradigma de programación establece un patrón o una forma en la que nuestros programas van a estar estructurados.

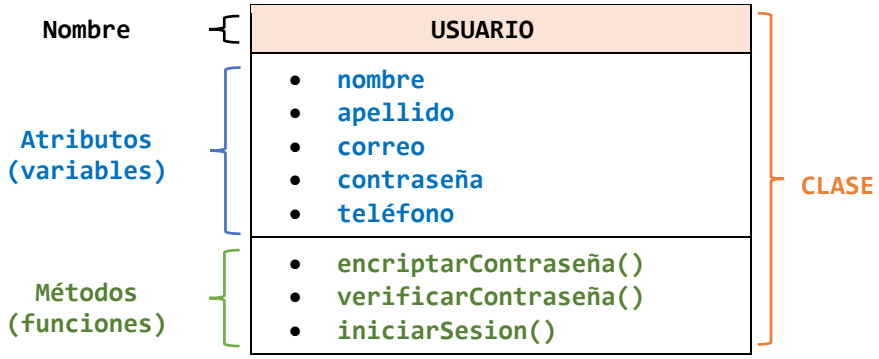
2. ¿QUÉ ES LA PROGRAMACIÓN ORIENTADA A OBJETOS (POO)?

Es un *paradigma de programación* que organiza el software en "objetos" que combinan datos O características (atributos) y comportamientos o funcionalidades (métodos), modelando elementos del mundo real.

CLASE = OBJETO
ATRIBUTOS = DATOS, CARACTERÍSTICAS
MÉTODOS = FUNCIONALIDADES, COMPORTAMIENTOS

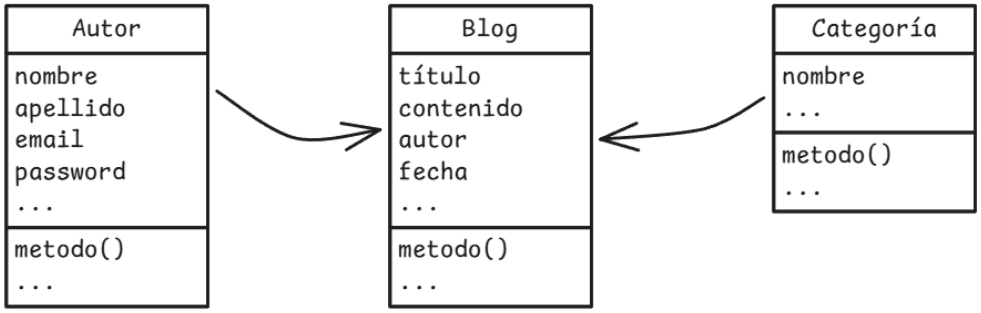
3. CLASES (PLANTILLA DE UN OBJETO)

Una Clase es la representación en código de lo que es un objeto. También es conocida como una plantilla o prototipo, de los cuales a partir de este se empiezan a crear distintas cosas o copias para usarlas de forma distinta, pero con la estructura que ya vienen definidas.



3.1. RELACIÓN ENTRE CLASES

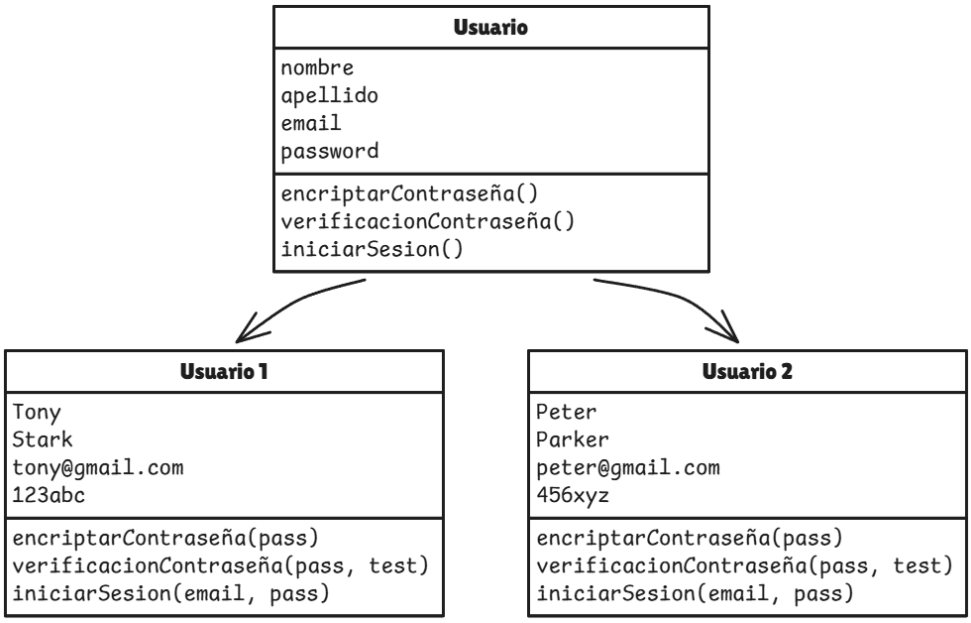
Los objetos (clases) se pueden relacionar unos con otros, es decir, podemos tener distintos objetos en nuestro software y estos pueden estar conectándose unos con otros para realizar ciertas funcionalidades o pueden estar trabajando de forma individual.



4. INSTANCIA DE UNA CLASE (OBJETO FUNCIONAL)

Cada **copia** que nosotros hagamos de una clase será un **objeto**, en programación *a estas copias se las conoce como instancias de una clase*.

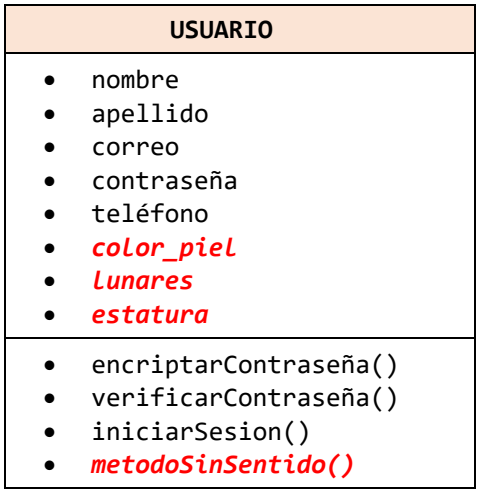
Una instancia es un objeto específico y funcional creado a partir de una clase. Funciona como una realización concreta de la "plantilla" (clase) que reside en la memoria, con sus propios valores únicos para los atributos, pero compartiendo la misma estructura y métodos definidos en la clase.



5. CUATRO PILARES DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

5.1. ABSTRACCIÓN

Es el proceso de identificar las características y comportamientos esenciales de un objeto del mundo real, ignorando los detalles complejos e innecesarios.

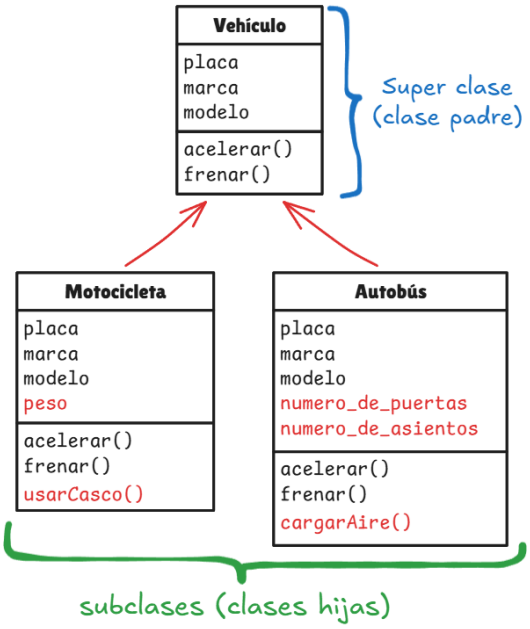


Una clase abstracta en Dart es una clase que no se puede instanciar directamente, marcada con la palabra clave `abstract`. Se utiliza para definir una interfaz común o un comportamiento base que otras clases (subclases) deben ***implementar o heredar***, permitiendo métodos abstractos (sin cuerpo) y concretos (con implementación).

Métodos Abstractos: Métodos sin implementación que las subclases deben sobrescribir `(@override)`.
Métodos Concretos: Puede contener métodos con código funcional que las subclases heredan.

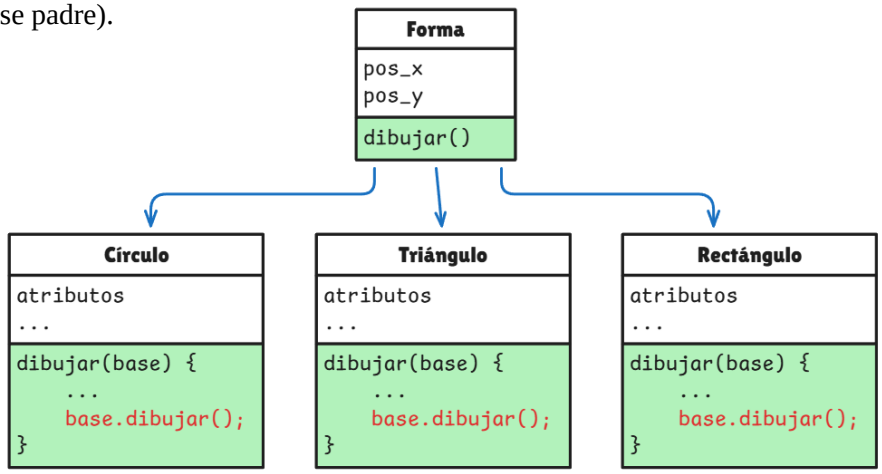
5.2. HERENCIA

Es un mecanismo que permite a una clase nueva (subclase o hija) heredar atributos y métodos de una clase existente (superclase o padre). Esto promueve la reutilización de código y establece una jerarquía "es-un" (ej. un Autobús es un Vehículo).



5.3. POLIMORFISMO

Es la capacidad que tienen objetos de diferentes clases para responder al mismo mensaje o *método* de formas distintas. Está muy relacionado con la herencia, ya que es la capacidad que tiene una subclase (clase hija) de cambiar el comportamiento que está de forma predeterminada dentro de los métodos de que tiene su super clase (clase padre).



5.4. ENCAPSULAMIENTO

Consiste en restringir el acceso directo a atributos y métodos internos de una clase para que no puedan ser modificados sin autorización, **protegiendo así la integridad del objeto**.

Utiliza modificadores de acceso (`private, public, protected`) y métodos `getters/setters`. Así *se expone sólo una interfaz pública para interactuar con la clase de manera controlada y segura*.

Clase
• **atributo 1 (privado)** • **atributo 2 (público)** • **atributo 3 (protected)**
• metodo1() (público) • metodo2() (público) • **metodo3() (privado)**

Acceso Controlado (Getters y Setters): Los getters y setters son *métodos públicos* especiales utilizados para acceder (`get`) y modificar (`set`) los atributos privados de una clase.

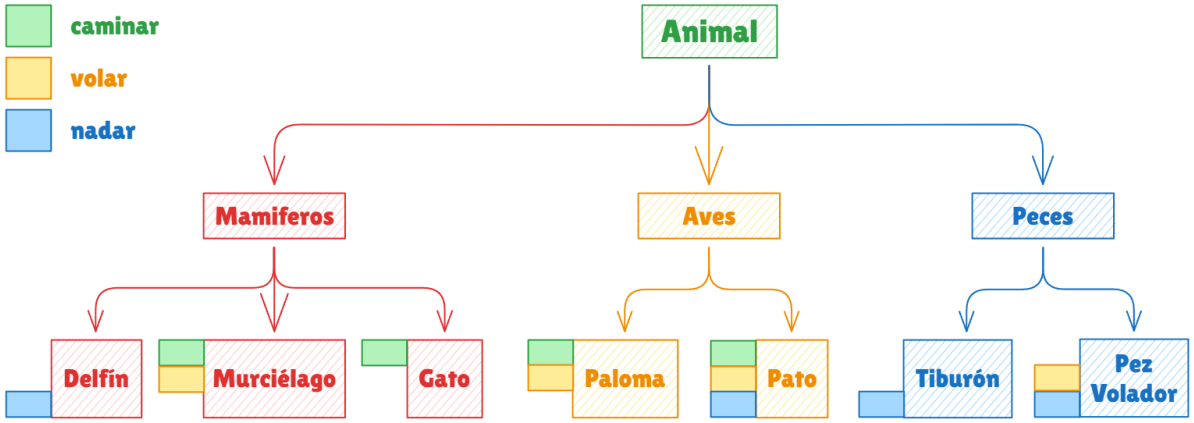
- **Getter (Accesor):** Método que devuelve el valor de una variable privada.
- **Setter (Mutador):** Método que establece o actualiza el valor de una variable privada, a menudo incluyendo lógica de validación.

6. MIXINS

Un **mixin** en Dart es una forma de reutilizar código de una clase en múltiples jerarquías de clases *sin usar la herencia tradicional*. Permite "inyectar" métodos y propiedades en varias clases usando la palabra clave **`with`**, facilitando la creación de código más modular, limpio y evitando la duplicación.

Características clave de los mixins:

- **Reutilización:** Permiten compartir comportamientos (funcionalidades) entre clases que no están relacionadas jerárquicamente.
- **Sintaxis:** Se declaran con la palabra clave **`mixin`** y se aplican con **`with`**.
- **Múltiples Mixins:** Una clase puede utilizar uno o más mixins (por ejemplo, class Gato extends Mamifero with Caminante, Nadador).
- **Restricciones:** No pueden tener constructores definidos.
- **Limitación de alcance:** Se puede usar la palabra clave on para limitar qué clases pueden usar el mixin.


caminar
volar
nadar
Animal
Mamiferos
Aves
Peces
Delfín
Murciélago
Gato
Paloma
Pato
Tiburón
Pez Volador